Softwares de Sistemas
Aula – Software de Sistemas

SOFTWARE BÁSICO, TURMA A

PROF. MARCELO LADEIRA – CIC/UNB

Módulo Softwares de Sistemas

Tópicos a serem vistos

- Arquitetura de von Neumann
- Código absoluto vs relocação estática
- Carregador absoluto
- Montador
- Ligador
- Carregador de relocação

Introdução

Softwares de sistema ou softwares básicos

- são programas que suportam a operação de um computador
  - são dependentes das arquiteturas das máquinas
- possibilitam que o usuário não precise entender o funcionamento interno da máquina
  - Fazem parte do conceito de computadores virtuais

Softwares de aplicação usam o computador como uma ferramenta e se preocupam com a solução de um problema

Introdução: exemplos de Softwares de Sistema

Tradutores

- Traduzem programas

Ligadores (linkers ou binders)

- Resolvem referências externas e preparam os programas para execução.

Carregadores (loaders)

- Carregam os programas em memória

Depuradores simbólicos (debbugers)

- Facilitam a identificação de erros

Montadores (assemblers)

- Montam programas a partir de código Assembly

Macro processadores

- Montam programas que possuem macros instruções

Sistemas Operacionais

- Gerenciam e controlam as máquinas

Arquitetura de von Neumann

CPU

- ULA (Unidade Lógica e Aritmética)
- UC (Unidade de Controle)
- Registradores
- PC (contador de programas)

Memória

- Dados e programas

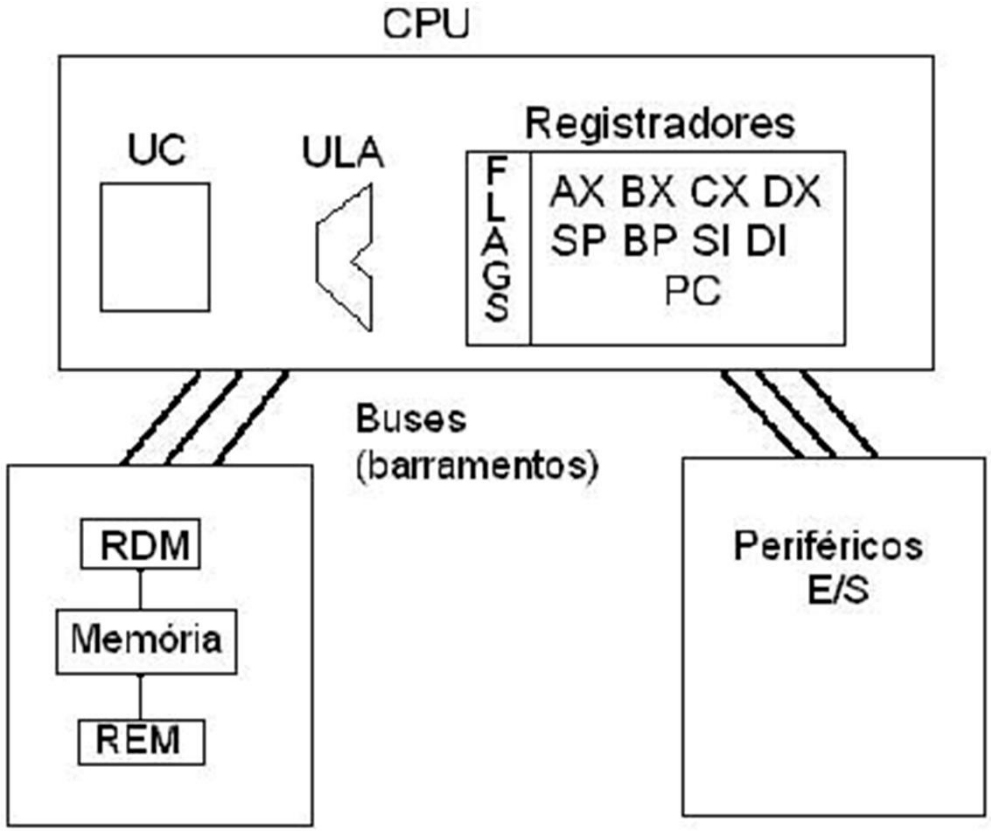
Periféricos de E/S

- Podem ser mapeados na memória

Barramentos

- Dados, endereço e controle

Arquitetura de von Neumann



Arquitetura de von Neumann: Características

Dados e programa armazenados na memória

- Instruções e dados só podem ser processados se estiverem na CPU

Processamento é seqüencial

- Ciclo busca – decodificação – busca de operandos- execução (fetch – execute cycle) e PC
  - PC ← Entry point
  - PC é incrementado a cada ciclo de busca
- Usamos linguagens imperativas porque usamos a arquitetura de Von Neumann
  - Função da metodologia de programação atual
    - Uso de programação modular e programação estruturada

Execução rápida porque dados e instruções são armazenados em blocos de memória adjacentes

- Localidade de referência

Gargalo de von Neumann

- Velocidade CPU >> velocidade de transferência da memória para a CPU

  Velocidade da memória >> velocidade de transferência dos dados dos periféricos de E/S para a memória ou CPU

Arquitetura de von Neumann

Redução do gargalo de von Neumann

- CPU vs memória
  - Memória cache
  - Processadores com múltiplos estágios (pipelining)
- Memória vs periféricos de E/S
  - Blocagem e bufferização
  - DMA (Acesso direto à memória)
  - Canais de controle de dispositivos de E/S

Introdução: Resolução de uma Operação de E/S

Laço de espera ocupado

- Queima CPU: E/S está finalizada?

Interrupção

- E/S gera chamadas aos serviços do S.O.
- Informação não está disponível em buffer
  - Processo é bloqueado em uma fila específica
- Gera chamada ao Supervisor
  - Cria programa para canal de E/S e controladoras
  - Operação é realizada no periférico adequado
- Canal gera interrupção
  - Controlador de interrupções desbloqueia processo

Processo é enfileirado na fila de “prontos”

Introdução: Redução do Gargalo de von Neumann

Disco é o fator limitante de um processo

- CPU e LAN esperam tempos enormes pela transmissão de dados do disco :
  - a memória na CPU opera na ordem de 10^{-9}s
  - LAN opera na ordem 100 Mbits/s = 10^{-8} s
  - o disco opera na ordem de 10^{-3}s

Técnicas são usadas para resolver esse problema: multiprogramação, *striping*, RAM disk e disk cache

- multiprogramação
  - Requer informação em disco: processo bloqueado.
  - O tempo da CPU é cotizado entre outros processos.
  - Processo bloqueado acorda ao receber sinal de interrupção.
    - Volta para a fila dos processos prontos (*ready*) a espera da CPU.

Introdução
Redução do Gargalo de von Neumann

Outra técnicas usadas para prover dados a CPU

- RAM disk
  - Uma parte da memória configurada para simular o comportamento de um disco mecânico, exceto a velocidade e não-volatilidade.
  - Esse recurso é usado, geralmente, para simular um disco flexível.
- Disk cache
  - Um bloco de memória é configurado para conter páginas do disco.
  - Quando dados do disco são necessários, a administração de disco procura no cache, antes de buscá-los em disco.
- *Striping*
  - Divisão do arquivo em fatias, alocando cada uma em um disco diferente e paralelamente enviando os dados para a LAN/RAM.

Redução do Gargalo de von Neumann

Administração de Buffer

Buffer é um trecho na memória de tamanho igual a um ou mais setores ou então blocos (registros físico), dependendo do contexto.

- O propósito do buffer é reduzir acessos em disco.

O sistema de E/S manter ao menos um buffer para cada arquivo aberto.

- Em mainframes é possível alocar vários buffer para um mesmo arquivo.

Gargalo de Buffer

- Se há apenas um buffer para cada arquivo (ou programa)
  - a leitura implica em preencher o buffer e acessar os registros componentes do buffer.
  - A leitura do próximo registro requer uma nova leitura física.
  - Mesmo se o programa transmite na mesma direção, um só buffer é moroso.
- Esse problema pode ser minorado com 2 ou mais buffer:
  - enquanto o S.O. lê um buffer o processador de E/S pode preencher outro.
  - enquanto o S.O. preenche um buffer o processador de E/S grava o outro.

Código Absoluto vs Relocação Estática

Código absoluto

- Posições fixas (e reais) de memória
- Endereços fixos (e reais) de memória
  - Memória ROM em procedimentos de boot.
    - No endereço FFFF:0000 da ROM BIOS tem uma instrução jmp para o endereço atual do código a ser executado.

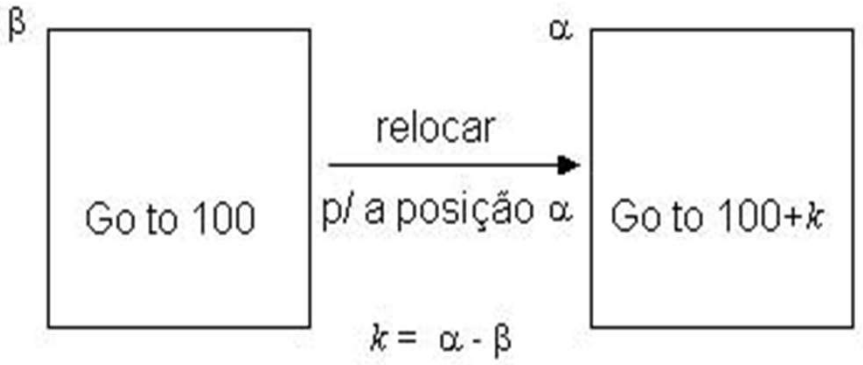
Existem situações onde não é prático definir o endereço onde um programa será carregado

- O tradutor precisa indicar quais as partes do programa que precisam ser modificadas
  - Um programa objeto que contém estas informações é um programa realocável
    - Instruções que usam endereçamento relativo não precisam ser modificadas

Código Absoluto vs Relocação Estática

Relocação estática

- O código pode ser executado em qualquer posição da memória
- Usa endereços relativos a uma base α
  - O endereço efetivo (absoluto) é dado por: offset + α
    - A relocação é obtida pela soma da constante de relocação aos endereços relativo, denotada k, constante de relocação.



Carregador Absoluto

Inicia a carga do sistema operacional

- Código absoluto
- Armazenado em memória não volátil, em posição fixa e conhecida (ROM, flash)

Ativação

- "Ação do operador ao apertar o botão" (liga/desliga ou específico)

Ação

```
pressload:
  read(for i:=0 step 1 until 79 do memory[i]);
transferto(memory[0]);
```

- Em grande porte (mainframe) o carregador absoluto é conhecido como IPL (*initial program load*)

Carregador Absoluto

Código do programa carregador absoluto

```
Load: read(for i:=0 step 1 until 79 do memory[r+i]);
                                /* r é o endereço do buffer utilizado */
LA:= memory[r];
n:=memory[r+1];
if n=0 then transferto[LA];
for i:=0 step1 until n-1 do memory[LA + i ] :=memory[r+2+i] goto
  load;
```

Layout do registro do programa a carregar

- Campo 1 – endereço de carga (load address)
- Campo 2 – número de palavras ou bytes a carregar desse registro
- Campo 3 – informação a ser carregada

Montador

Programa que converte um programa *Assembly* em um código objeto relocável (código de máquina) se o programa *Assembly* estiver correto.

Entrada

- Arquivo texto com código *Assembly*

Saída

- Arquivo binário com código de máquina relocável
- Tabela de endereços relocáveis (ou lista)
- Lista de referências externas (símbolos não resolvidos pelo montador)

Função básica

- Verificação de sintaxe
- Resolução de endereços simbólicos
- Tradução de mnemônico em código de máquina
- Tratamento de macros

SE HOUVER ERRO DE SINTAXE, O MONTADOR NÃO GERA O CÓDIGO E EMITE UMA MENSAGEM DE ERRO.

Montador: Estruturação

Passo único (ler o fonte apenas uma única vez)

- Monta tabela de símbolos
- Verifica sintaxe
- Gera código objeto relocável
- Expande macros e diretrizes de montagem

➔ Maior complexidade devido ao tratamento de "referência futura"

Múltiplos passos (múltiplas leituras do código)

- Pré-passo:
  - trata macros e diretrizes de montagem
- Passo 1:
  - monta tabela de símbolos
  - Verifica sintaxe e semântica
  - Monta código intermediário
- Passo n:
  - Monta código intermediário otimizado
- Passo final:
  - gera código objeto relocável

QUAL A RAZÃO PARA MAIOR COMPLEXIDADE NO TRATAMENTO DE REFERÊNCIAS FUTURAS? COMO TRATAR REFERÊNCIAS FUTURAS? QUAL O NÚMERO DE PASSOS CARACTERÍSTICO ATUALMENTE? PORQUÊ?

Montador
Principais Estruturas de Dados

Contador de programa (PC)

- Utilizado para simular a alocação de memória
  - Aponta para o próximo endereço relativo a ser alocado.
    - Iniciado com o valor zero ou o especificado na diretiva org (ou equivalente) do Assembly.
  - Após o processamento de cada sentença no fonte, o tamanho da instrução ou da área de dados alocada é adicionado a PC.
  - Quando se encontra no fonte a definição de um símbolo, o valor de PC indica o endereço a ser associado a ele.
- Sugestão: inteiro longo
  - Representa um *offset* em relação a um endereço base.

Montador
Principais Estruturas de Dados

Tabela de Símbolos

- Armazena símbolos internos (locais) ou externos representando nomes de variáveis, procedimentos, rótulos e constantes simbólicas.
  - Estruturação típica
    - Identificador
    - Tipo (constante simbólica, byte, word, double word, rótulo, macro, segmento, grupo, procedimento)
    - Valor (endereço)
    - Valor inicial
    - Local vs externo
    - Declarado vs referenciado
  - Sugestão: árvore binária de pesquisa

ALGUNS MONTADORES INCLUEM OS NOMES DOS REGISTRADORES NA TABELA DE SÍMBOLOS.

Montador
Principais Estruturas de Dados

Tabela de Mnemônicos

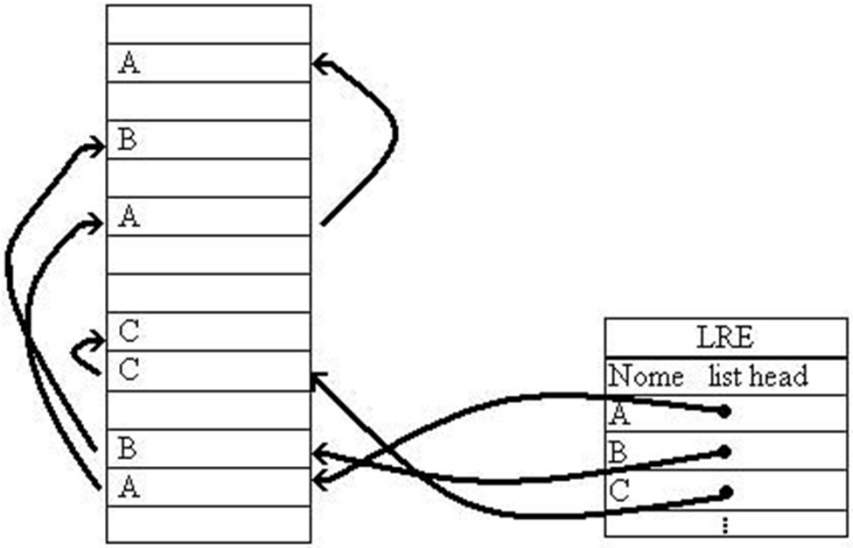
- Descreve a sintaxe dos mnemônicos e o código de operação
- Estruturação Típica
  - Mnemônico
  - Tipo (instrução, pseudo-instrução)
  - Rótulo: (obrigatório, opcional, não permitido)
  - Número de operandos
    - Tipos dos operandos
    - Código de operação
    - Número de bytes
- Sugestão: vetor ordenado pelos mnemônicos e vetor de ponteiro para funções “verifica sintaxe”

OS 4 PRIMEIROS CAMPOS PODEM SER ASSOCIADOS AO VETOR ORDENADO. OS 3 ÚLTIMOS CAMPOS PODEM SER ESPECÍFICOS PARA CADA TIPO DA INSTRUÇÃO E PODEM SER ASSOCIADOS À FUNÇÃO "VERIFICA SINTAXE" CORRESPONDENTE.

Montador
Principais Estruturas de Dados

Lista de referências externas

- Contém os símbolos só referenciados mas não declarados no programa
  - são externos.
- Lista encadeada reversa que será utilizada pelo ligador para resolver as referências externas.



UMA OUTRA ABORDAGEM É ASSOCIAR A CADA SÍMBOLO EXTERNO UM ENDEREÇO LOCAL ONDE SERÁ ARMAZENADO O ENDEREÇO REAL DO SÍMBOLO, QUANDO ESTE FOR CONHECIDO DURANTE O PROCESSO DE LIGAÇÃO.

Montador
Principais Estruturas de Dados

Lista de endereços relocáveis

- Lista de variáveis ou procedimentos ou rótulos.
  - É utilizada para relocação estática pelo ligador e pelo carregador de relocação.
- É a existência da relocação que possibilita o uso de bibliotecas de código objeto.

Montador
Principais Estruturas de Dados

Tabela de macros

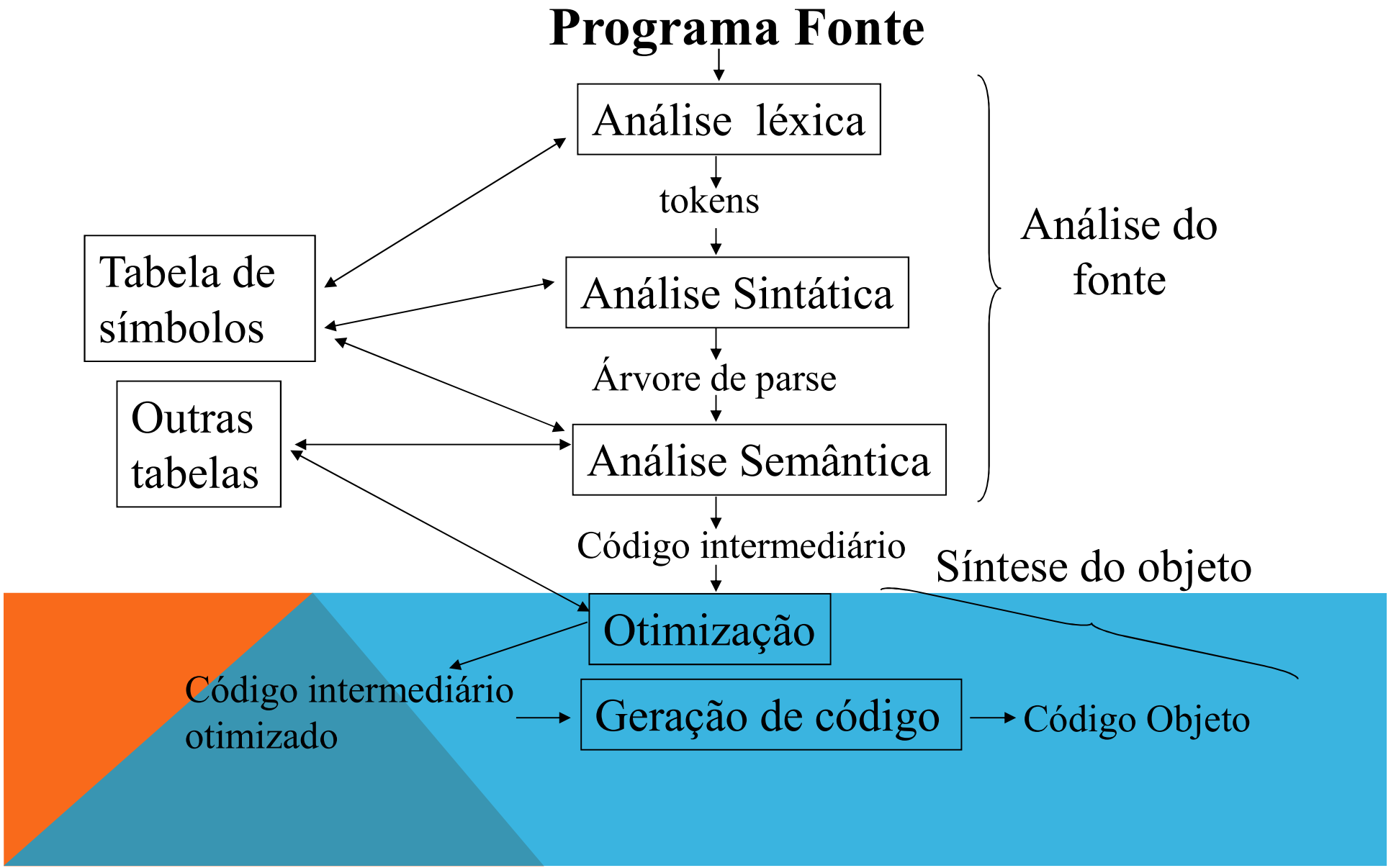
- Contém:
  - Nome da macro
  - Numero de parâmetros (argumentos simbólicos)
  - Ponteiro para a string associada à macro.

Tabela de mensagens de erro

Indicador de ocorrência de erro

Indicador de variável, rótulo ou procedimento referenciado, mas não declarado.

Montador: Estrutura Lógica



Montador
Estrutura Lógica: Pré-passo

Passo inicial no processo de montagem

- Função
  - executar diretivas que alteram o programa fonte, criando flexibilidades para o programador e facilitando as outras etapas do processo de montagem.
    - Montagem condicional, inclusão de arquivos, substituição de macros por string, tratamento de macros com parâmetros.
- Entrada
  - Código fonte em *Assembly* contendo diretivas de montagem.
- Saída
  - Código fonte limpo em *Assembly*
    - sem comentários ou delimitadores redundantes e expansão de macros.
    - Esse código limpo é entrada para o próximo passo do processo de montagem.

Estrutura Lógica: Pré-passo
Macros e Diretivas de Montagem

Processamento de macros e diretivas de tempo de Montagem

- declaração:
  - coloca na TS identificador que representa uma string de caracteres, cujo valor é um ponteiro para a string.
    - Macro Simples (sem argumentos).
    - Macro com parâmetros
      - Paramentos não são inseridos na TS porque não possuem tipo.
- expansão:
  - substitui, durante o pré-processamento, a ocorrência do identificador da macro no programa, pelo texto associado.

Montador
Estrutura Lógica

Análise léxica

- Grupa seqüência de caracteres em constituintes elementares do fonte (símbolos):
  - identificadores, delimitadores, operadores, números, palavras chaves, palavras opcionais, ...
- Identifica os itens léxicos (tokens) e os repassa aos outros estágios do tradutor.
- Identifica o tipo de cada token e lhe anexa um rótulo de tipo.
- Converte números para sua representação binária (inteiro e reais).
- Inclui identificadores na TS ou substitui referências ao identificador (já incluso na TS) pelo endereço dado pelo semântico.

O FONTE É ARQUIVO TEXTO, PORTANTO UMA SEQUÊNCIA DE CARACTERES, SEM NENHUMA ESTRUTURAÇÃO EXCETO A QUEBRA DE LINHA.

Montador
Estrutura Lógica

Análise sintática

- Identifica as unidades sintáticas do programa:
  - comandos ou declarações, usando os itens léxicos.
    - Verifica se formam uma estrutura válida
- Chama o analisador semântico para processar essa unidade.

Montador
Estrutura Lógica

Análise semântica

- Verifica se a estrutura é válida, tendo em vista o contexto.
- Mantêm a TS.
- Insere informações implícitas (uso de *defaults*).
  - Largura de constante
    - MOV AX, 10h ; armazena 16 com word (2 bytes)
- Gera código intermediário
  - Passado para a fase de Síntese.
  - Geração é interrompida se ocorrer um erro.

Montador
Estrutura Lógica

Síntese

- Otimiza código (opcional)
  - Otimiza o código intermediário gerado pela análise semântica.
    - A otimização é do código e não das funções implementadas ao nível do código do programador
- Gera código objeto relocável.

ESSA FASE SOMENTE SERÁ EXECUTADA SE NÃO OCORRER ERROS NA FASE DE ANÁLISE.